



Desarrollo Reactivo con Spring Boot

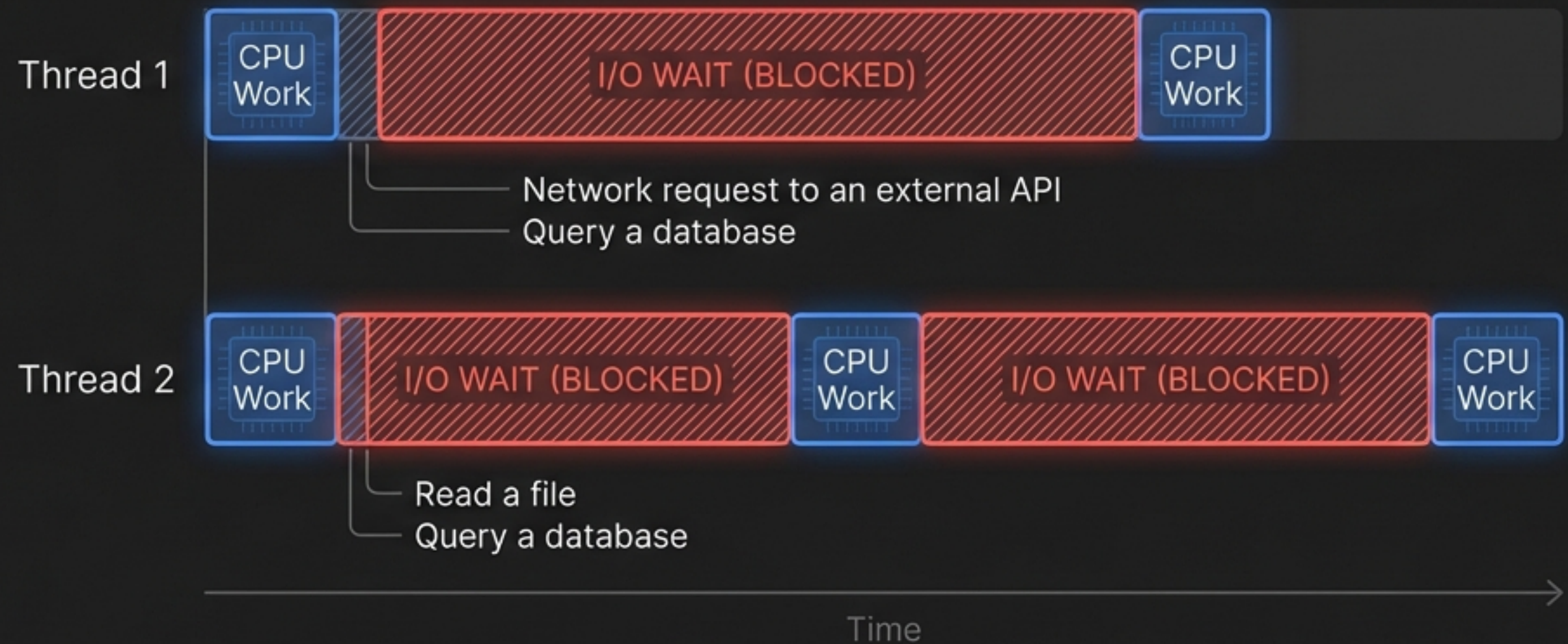
Arquitectura No Bloqueante, Project Reactor y Flujos de Datos

JetBrains Mono
Curso de Microservicios Reactivos

El Cuello de Botella: I/O Bloqueante

El **carta de Gantt** en la delotil de ternficos datas que perman a los bloques, esperando un comtamente con JetBrains Mono por JetBrains Mono.

- **Modelo Thread-per-Request:** Un hilo por petición.
- **El Problema:** CPU ocioso esperando a la Base de Datos.
- **Consecuencia:** Agotamiento de hilos y alto consumo de memoria.

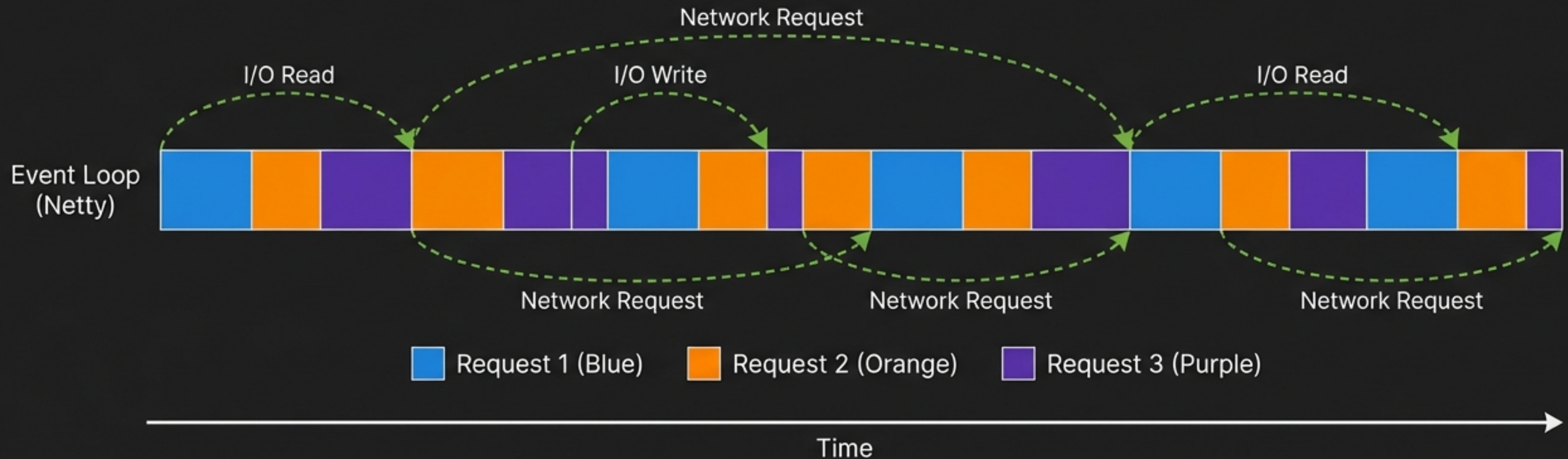


La Revolución: Event Loop No Bloqueante

Arquitectura Reactiva: Un solo hilo (Event Loop) maneja todo.

Funcionamiento: Inicia tarea -> Pasa a la siguiente -> Callback al terminar.

Resultado: Máxima concurrencia con mínimos recursos.



Project Reactor: El Motor de Spring WebFlux



Netty
(Event Loop)

JetBrains Mono

+



Project Reactor
(Core Lib)

JetBrains Mono

=



Spring
WebFlux

JetBrains Mono

Implementación de Reactive Streams (Publisher/Subscriber)

JetBrains Mono

Los Tipos Base: Mono vs. Flux

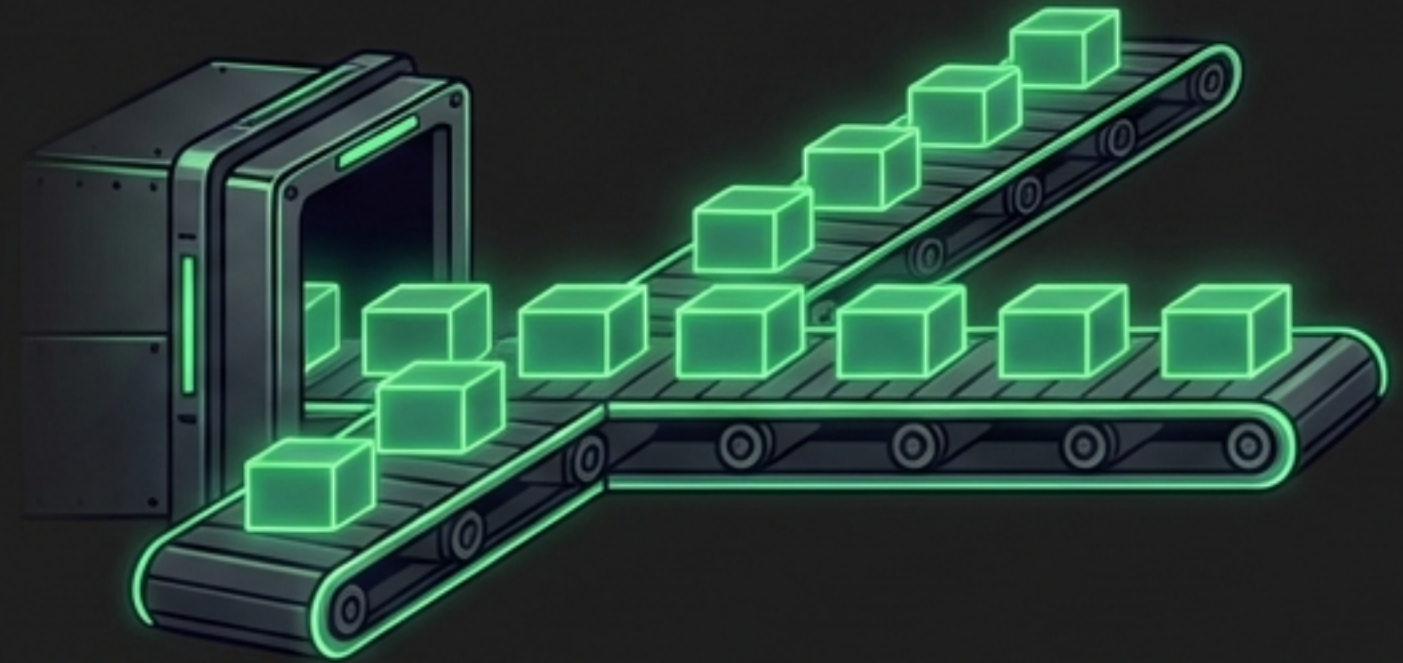
Mono<T>



0 o 1 Elemento

Ideal para: `findById()`, `ResponseEntity`

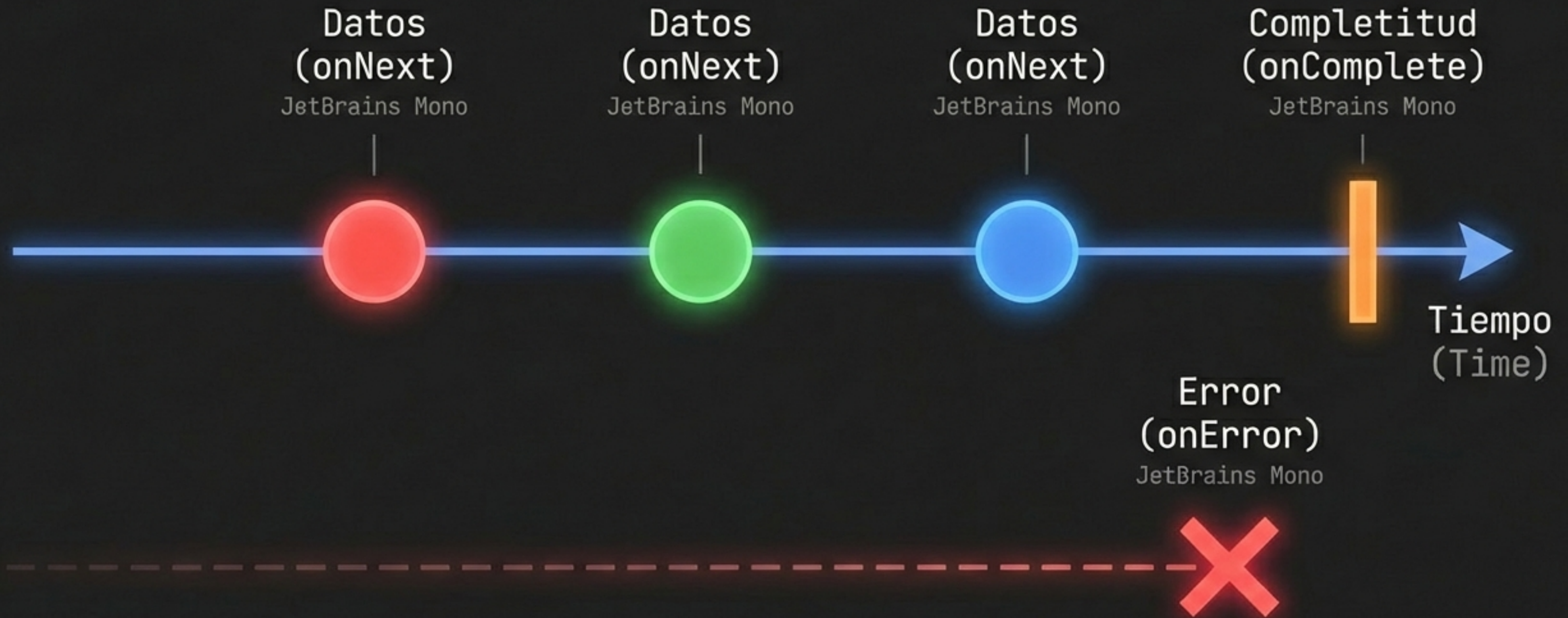
Flux<T>



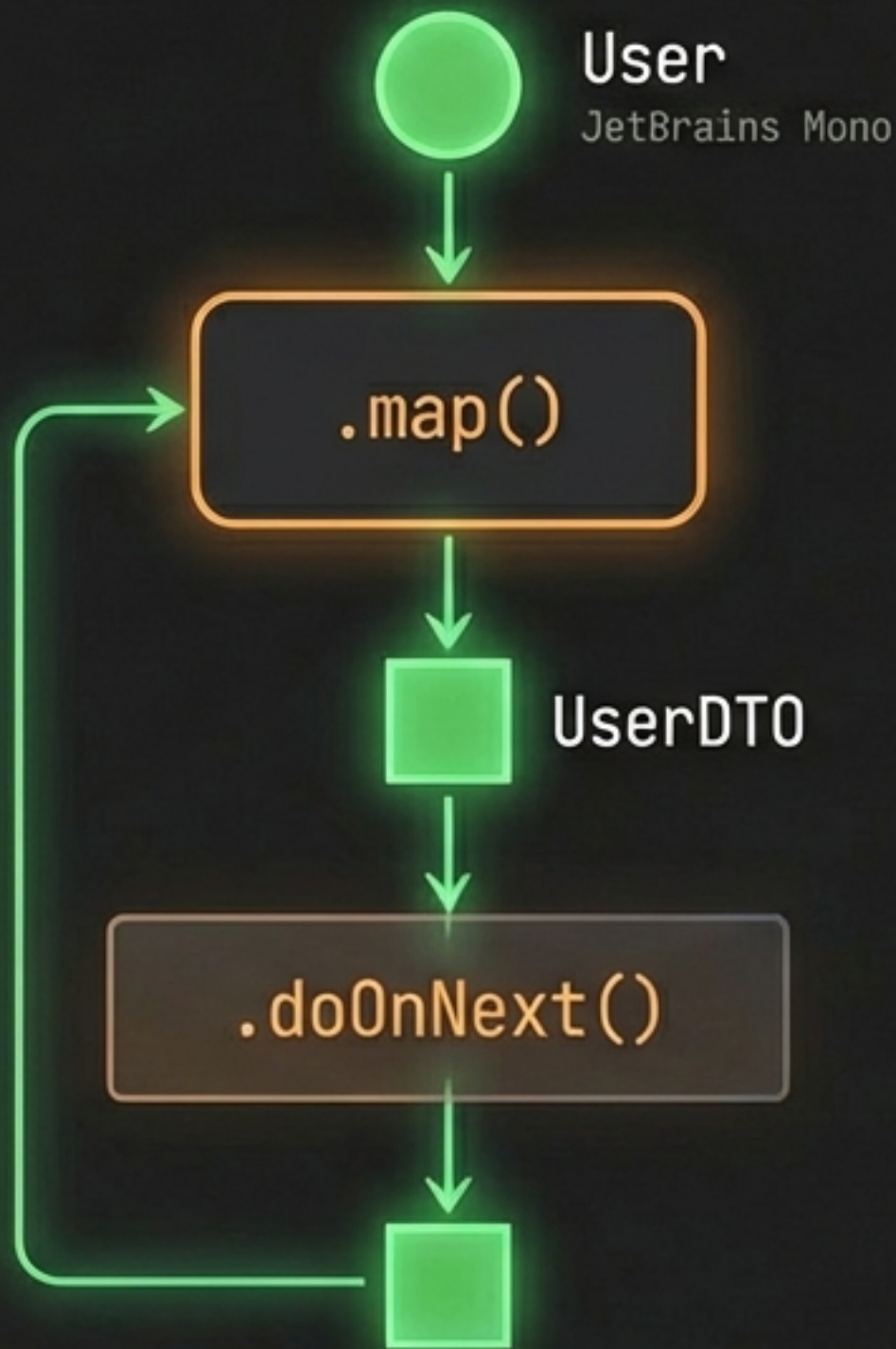
0 a N Elementos

Ideal para: Listas, Streams Infinitos, Kafka

Entendiendo los Diagramas de Canicas



Operadores: La Línea de Montaje



```
Flux.fromIterable(users)
    .map(u -> u.toUpperCase()) // Transformación
    .doOnNext(u -> log.info(u)) // Efecto Secundario
    .subscribe(); // Inicia el flujo
```


Spring WebFlux: API REST Reactiva

Imperativo (Blocking)

```
public User get(@PathVariable String id)
```



Entrega Directa de Datos

Reactivo (Non-Blocking)

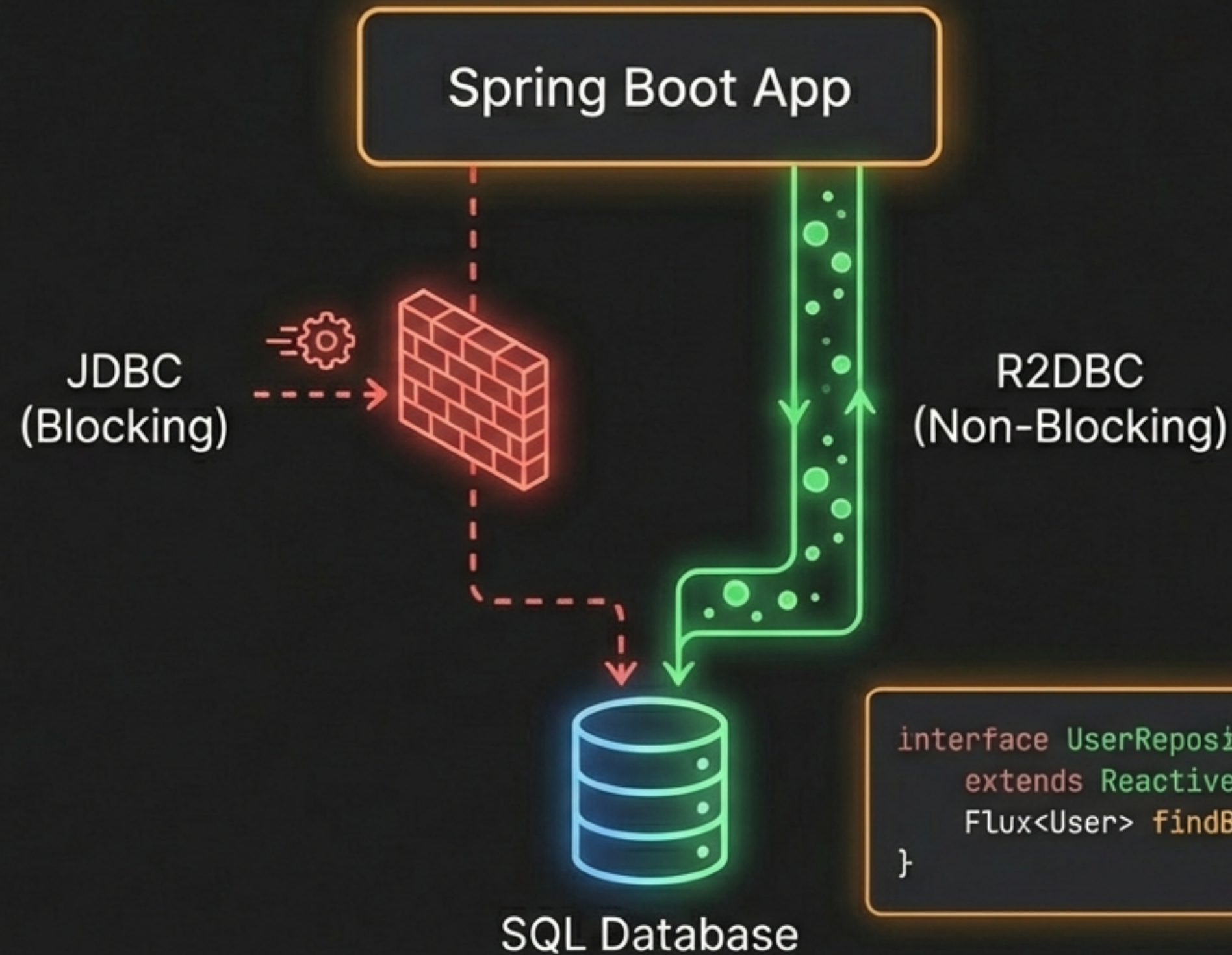
```
public Mono<User> get(@PathVariable String id)
```



Entrega de Promesa (Mono/Flux)

No retornamos el dato. Retornamos la promesa del dato.

Persistencia Reactiva con Spring Data R2DBC



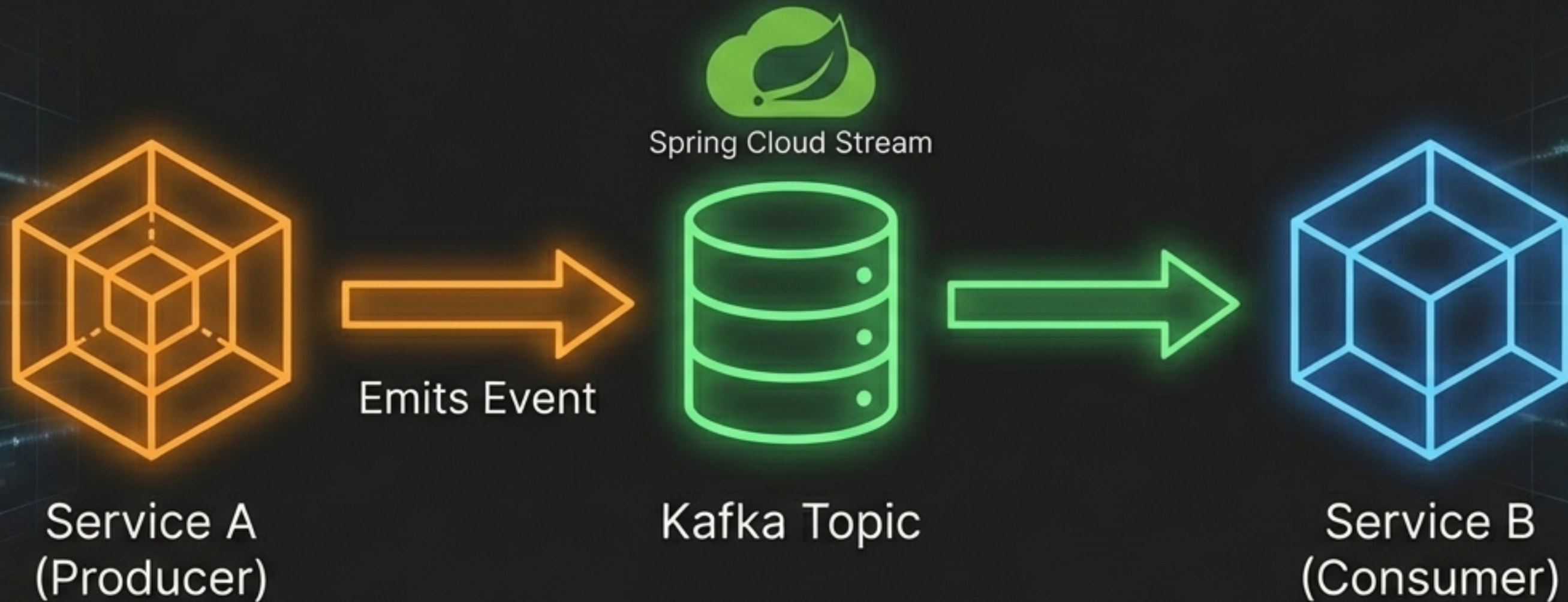
```
interface UserRepository
    extends ReactiveCrudRepository<User, Long> {
        Flux<User> findByName(String name);
    }
```


Backpressure: Control de Flujo



El Suscriptor decide cuántos datos puede procesar para evitar OutOfMemory.

Arquitectura Orientada a Eventos



Desacoplamiento total: Los servicios se comunican a través de flujos de datos, no peticiones HTTP directas.

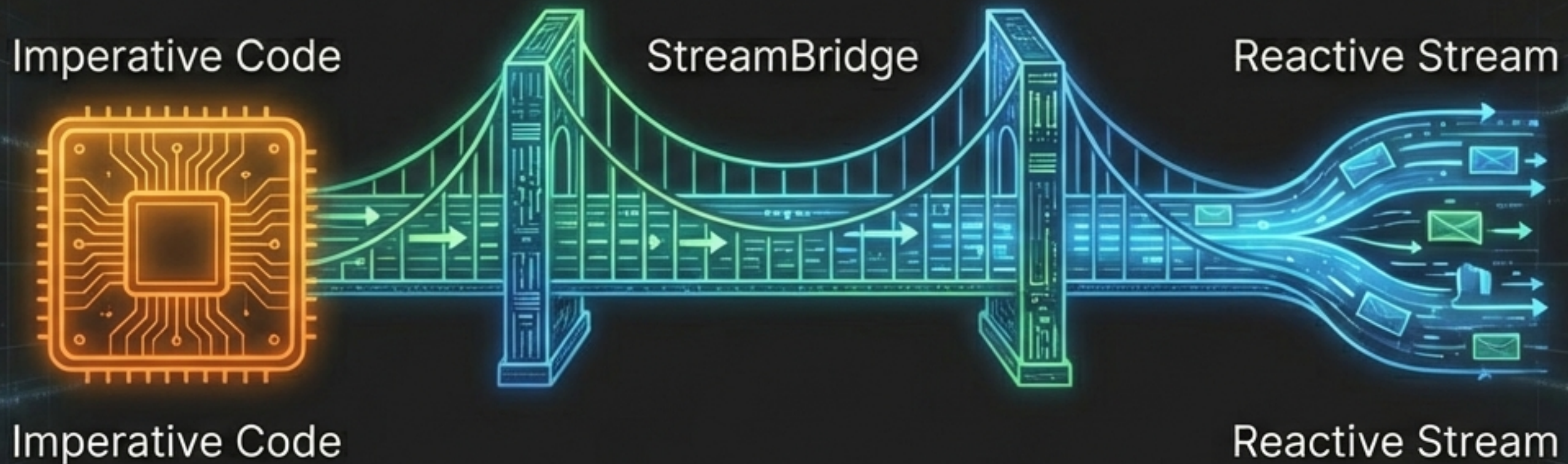
Consumo de Eventos (Consumer)



```
@Bean
public Consumer<Flux<String>> consumeEvents() {
    return flux -> flux
        .doOnNext(evt -> log.info(evt))
        .subscribe();
}
```

Reference: Similar to Quarkus @Incoming logic, but using Reactor Flux.

Producción de Eventos (Producer)

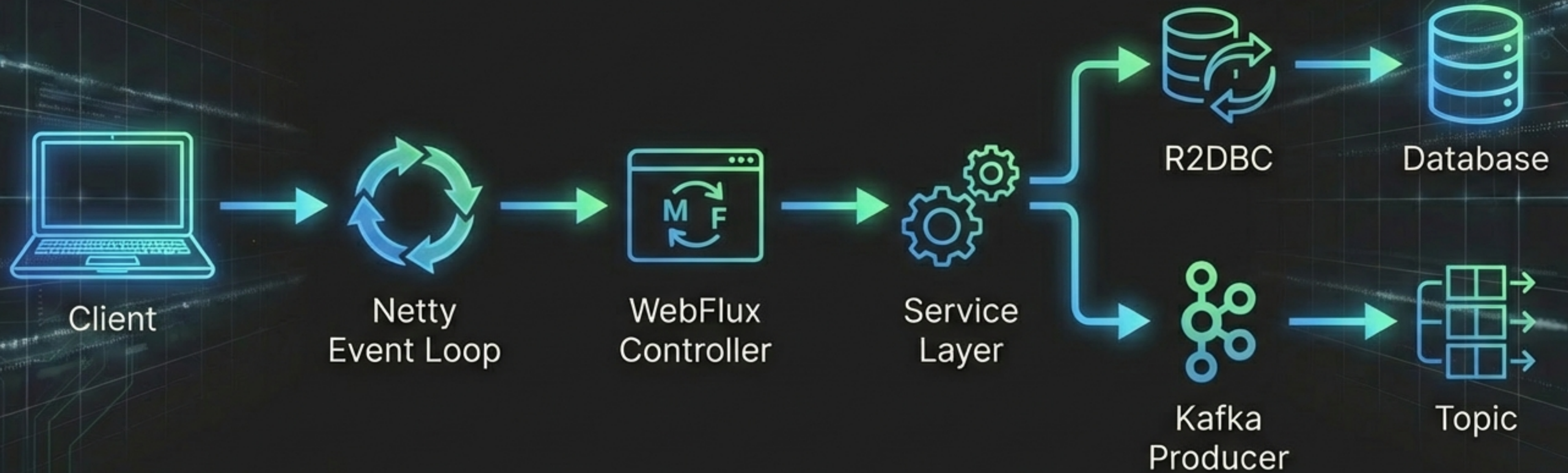


```
// Envío desde código imperativo  
streamBridge.send("users-out", new UserCreatedEvent(id));
```

Conectando el mundo imperativo con los streams reactivos.

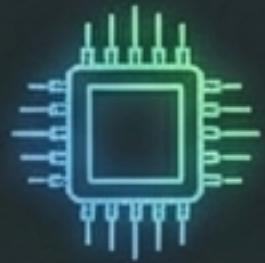
Reference: Similar to the outgoing emitter concept in <IMAGE_2>.

El Stack Reactivo Completo



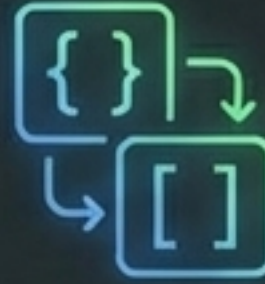
Reactividad de extremo a extremo: Desde el navegador hasta la infraestructura.

Resumen: El Cambio de Paradigma



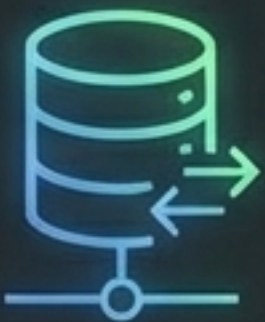
Threads

De Thread-per-Request
→ a Event Loop.



Tipos

De Listas a
Flux / Mono.



DB

De JDBC a
R2DBC.



Mensajería

De HTTP a
Kafka Streams.



Project Reactor Guide

Piensa en Flujos, no en Pasos. ↕